

박영진 — Backend Lead 회고 발표 대본

슬라이드 18 / 예상 발표 시간: 3~5분 마인드맵 브랜치를 hover하며 각 주제별로 발표

도입 (30초)

안녕하세요, 백엔드 리드를 담당한 박영진입니다.

저는 MSA 전체 설계, 인증과 Gateway 서비스, CI/CD 파이프라인, 그리고 file-service의 Go 전환을 담당했습니다.

11주간 510건의 커밋과 17회의 릴리즈를 거치면서 겪은 아키텍처 설계, 장애 대응, 그리고 기술적 성장에 대해 이야기하겠습니다.

1. MSA 설계 (hover: MSA 설계 브랜치)

프로젝트 초기에 모놀리식으로 시작했지만, 1주차에 MSA로 전환하는 결정을 내렸습니다.

전환 계기는 video-service의 LiveKit 의존성이 auth-service의 빌드를 느리게 만들고, ai-service의 무거운 STT 처리가 다른 서비스 응답 시간에 영향을 주는 상황이었습니다.

7개 서비스를 도메인 경계로 분리하고, 서비스 간 통신은 Kafka 이벤트 버스로 결합도를 낮췄습니다. MSA 덕분에 **서비스별로 최적의 기술을 선택**할 수 있었고, file-service를 Java에서 Go로 전환한 것이 대표적인 사례입니다.

2. 인증 & Gateway (hover: 인증 & Gateway 브랜치)

MSA에서 가장 먼저 부딪힌 문제는 "각 서비스가 어떻게 JWT를 검증할 것인가"였습니다.

모든 서비스에 비밀키를 공유하면 한 서비스가 뚫렸을 때 전체가 무너집니다. 그래서 **JWKS 표준**을 채택했습니다. auth-service만 RSA 개인키를 보유하고, `/.well-known/jwks.json`으로 공개키를 노출합니다. Gateway가 공개키로 JWT를 검증하고 사용자 정보를 헤더에 담아 전달하는 구조입니다.

JWT는 **HttpOnly Secure 쿠키**에 저장해서 XSS 토큰 탈취를 원천 차단했고, CORS는 Gateway에서만 중앙 관리하여 헤더 중복 문제도 해결했습니다.

3. Go 전환 (hover: Go 전환 브랜치)

Java/Spring Boot 기반 file-service는 단순한 파일 업로드/다운로드에 **메모리 512MB, 응답 200ms**로 과도했습니다.

Go(Gin + GORM)로 전면 재작성한 결과:

- API 응답: **200ms → 2ms** (100배 개선)
- 메모리: **512MB → 50MB**
- Docker 이미지: **300MB → 20MB**

MSA 아키텍처 덕분에 다른 서비스에 영향 없이, file-service만 독립적으로 기술 스택을 교체할 수 있었습니다. 이것이 MSA의 진짜 장점이었습니다.

4. 장애 대응 (hover: 장애 대응 브랜치)

운영 중 겪은 세 가지 주요 장애가 있었습니다.

첫째, **Nginx DNS 캐시 502**. Gateway 컨테이너가 재시작되면서 Docker 내부 IP가 바뀌었는데, Nginx가 이전 IP를 캐시하고 있어서 전체 API가 502로 멈췄습니다. CI/CD에 Gateway 배포 후 Nginx reload를 자동화해서 해결했습니다.

둘째, **이메일 3중 장애**. Gmail Refresh Token 만료, SMTP 호스트가 localhost로 남아있던 것, STARTTLS가 Spring Boot 자동 구성에 미인식된 것이 동시에 겹쳤습니다. 기술 스택 마이그레이션 시 설정 파일 전수 점검이 필수라는 교훈을 얻었습니다.

셋째, 가장 인상적이었던 것은 **"CORS 에러가 진짜 CORS 문제가 아닐 때"**입니다. notification-service DB 커넥션 풀이 고갈되면서 504 응답이 발생했는데, 504에 CORS 헤더가 없어서 브라우저가 CORS error로 표시했습니다. 한 서비스의 장애가 Gateway를 통해 전체로 전파된 사례였습니다.

5. CI/CD & 인프라 (hover: CI/CD & 인프라 브랜치)

7개 서비스를 매번 전부 빌드하면 시간이 낭비됩니다. **변경된 서비스만 감지해서 빌드/배포**하는 파이프라인을 설계했습니다.

Jib을 선택한 이유는 Dockerfile 없이 Gradle 플러그인만으로 Docker 이미지를 빌드하고, 레이어 캐싱이 기본 내장이기 때문입니다.

11주 동안 17회 릴리즈를 수행하면서 파이프라인에 지속적으로 개선이 추가되었습니다. 510건의 커밋, 각 커밋마다 "왜 이렇게 했는가"에 대한 답을 가지고 있다는 것이 가장 중요한 성과입니다.

마무리 (30초)

이 프로젝트에서 가장 크게 성장한 부분은 **장애를 통해 아키텍처의 의미를 체감한 것**입니다.

교과서에서 "서비스 간 장애 격리"를 읽었을 때는 막연했지만, DB 커넥션 풀 고갈이 전혀 다른 서비스까지 연쇄 장애를 일으키는 것을 직접 경험하고 나서야, "왜 서비스 간 타임아웃이 중요한지"를 진짜로 이해하게 되었습니다.

코드뿐 아니라 설정 파일, 환경변수, 인프라 구성까지 "배포 가능한 코드"의 범위에 포함된다는 것을 체감한 프로젝트였습니다.